

AI-Native Multicast Routing in FANETs using Graph Neural Networks and Deep Hierarchical Reinforcement Learning for Disaster Recovery Video Streaming

Nitish M

Computer Science Engineering
St. Joseph's Institute of Technology
Chennai, India
mnitish1705@gmail.com

Padmesh V A

Computer Science Engineering
St. Joseph's Institute of Technology
Chennai, India
vapadmesh@gmail.com

Dr. V Sabaresan

Computer Science Engineering
St. Joseph's Institute of Technology
Chennai, India
sabaresanvenugopal@gmail.com

Abstract—The problem of routing in Flying Ad Hoc Networks (FANETs) is qualitatively distinct from that in terrestrial mobile ad hoc networks (MANETs) mainly due to high-speed three-dimensional mobility, unpredictable and frequent changes in network topology, and the need for energy and delay constraints. Existing MANET routing protocols and bio-inspired optimizers such as Artificial Bee Colony (ABC/AABC) meet some of these routing goals but are not suited to learn online from the ultra-dynamic nature of UAVs. In this paper, we propose a Hierarchical Deep Reinforcement Learning (HDRL) and a Graph Neural Network (GNN) model for energy-efficient multicast routing in FANETs. Every UAV employs a hierarchical 2-level controller (Meta and Intrinsic) that takes 8-dimensional node features (location, velocity, energy, traffic, queue) and 4-dimensional edge features (signal strength, delay, loss, bandwidth) as input. A 2-layer GCN outputs 16-dimensional, graph-based features to build and maintain multicast trees. The Meta controller chooses optimal relay zones while the Intrinsic controller performs local path construction with explicit loop prevention (visited sets) and a maximum of 15 hops. Our approach includes an adaptive reward explicitly considering packet delivery ratio, delay, and energy efficiency with hop-level reward sharing. We evaluate it in a simulation framework that integrates ns-3 and our own Gym environment to generate 50 UAVs in a $2000 \times 2000 m^2$ region and train PyTorch HRL models. We show that the proposed approach achieves higher packet delivery ratio, reduced end-to-end delay, and prolonged network lifetime in high-mobility scenarios. Our GNN-HDRL multicast approach provides a feasible and flexible routing perspective for FANETs where both speed and energy efficiency are at stake.

Index Terms—Flying Ad Hoc Networks, FANET, Multi-Agent Deep Reinforcement Learning, Multicast Routing, Energy-Aware Routing, Centralized Critic.

I. INTRODUCTION

Cooperating unmanned aerial vehicles (UAVs) to form Flying Ad Hoc Networks (FANETs) are increasingly being deployed in disaster management, environment monitoring, precision farming and military communications. FANETs are three-dimensional, have faster node mobility, and more frequent network disruptions compared to MANETs. This has a large impact on route lifetime and makes multicast delivery to

multiple receivers difficult. Existing MANET protocols such as AODV and OLSR, as well as swarm-inspired optimizers like variants of the Artificial Bee Colony algorithm, can offer basic support, but may be unreliable in the face of rapid node dynamics, and learn to balance the trade-offs among conflicting QoS metrics in real time. Literature surveys on FANET routing typically highlight these shortcomings, and recommend more adaptive, learning-based approaches that build on local observations to make better forwarding decisions [1], [2].

We present an energy-efficient Hierarchical Deep Reinforcement Learning (HDRL) approach, backed by Graph Neural Networks (GNNs), for FANET multicast routing. Instead of fixed route discovery or periodic flooding, we propose a two-tier hierarchy which learns to choose next hops that jointly optimise long-term multicast throughput while minimising delay and energy consumption. We employ a Meta and Intrinsic controller framework with large experience replay buffers (10,000 and 20,000 transitions) which helps stabilize learning, while enabling each UAV to use low-dimensional (16-D) local GNN-based representations at run-time. The rationale for this decomposition is that the time scale of strategic (where to route through) and tactical (which neighbor to route to) decisions are different and a flat policy finds it difficult to learn to make both types of decisions.

Our work is divided into: (1) a formulation of multicast routing in 3D FANETs as a hierarchical Markov decision process, with a multi-objective reward function balancing delivery, latency and energy via explicit hop-level reward sharing; (2) a Graph Neural Network (GCN) representation of the state with 8 node-based and 4 link-based features; (3) an end-to-end HRL training framework supporting 50 UAVs with training, inference and evaluation modes; and (4) simulation results showing improvement over baseline routing and classical MANET protocols.

II. LITERATURE SURVEY

There is considerable literature on FANET routing. According to survey papers, the challenges are mainly related to the instability of routes due to highly dynamic movement, energy constraints, and the need for three-dimensional mobility models and link quality estimation models [1], [2]. Suggested solutions range from topology-based protocols inherited from MANETs to hierarchical and geographic routing protocols that are specially designed for FANET deployment patterns. Swarm intelligence and bio-inspired metaheuristics, such as Artificial Bee Colony (ABC) and its adaptive extensions, have been used for MANET routing, with goals such as shortest path routing or maximizing remaining energy of routes [5]. Such approaches can quickly find solutions for a reasonably stable network but they need to be regularly re-optimized due to network topology changes, limiting their responsiveness in rapidly changing FANET environments where link lifetimes are on the order of seconds.

Reinforcement learning (RL) and multi-agent deep RL (MADRL) has been applied to networking problems that require online learning and adaptation [9], [10]. Existing research shows MADRL for task offloading, scheduling, and delay-sensitive resource sharing. Other works explore MAPPO and MADDPG variants for intra-FANET decision tasks such as task scheduling and computation offloading [3], [4]. These studies demonstrate the ability of MADRL to cope with the combinatorial and non-stationary nature of FANET routing problems when combined with suitable state encoding and training strategies. Graph Neural Networks (GNNs) have also been used for routing in dynamic wireless networks, where the size of neighborhood changes across mobile networks, but the permutation invariant aggregation of GNNs helps in this setting [?]. The use of GNN state representations and hierarchical RL overcomes two key limitations in previous works: the inability to succinctly represent global topology information and the difficulty of learning policies that are stable across multiple time horizons. This motivates the multicast routing solution in this paper.

III. SYSTEM ARCHITECTURE

We develop a modular system with a layered architecture that separates the simulation and control, agent learning, and packet-forwarding layers. This enables the system to be tested at each layer and for the learned policies to be replaced without modifying the underlying network emulator. The system architecture flow chart is shown in Figure 1.

A. Network and Simulation Layer

This layer describes the FANET physical and MAC layer protocol behaviour during training and testing. The simulation environment sets up 50 UAVs uniformly spread over a $2000 \times 2000 m^2$ geographic area with a multicast group of 1 source and 5 destinations. UAVs follow a 3D Random Waypoint mobility model with explicit velocity state tracking to ensure that agent observations include the latest kinematic state. Radio link is defined by 4 properties: power level,

propagation delay, loss rate and bandwidth. The layer is written in Python and provides a Gym API, enabling standard RL tools and custom training loops to access it via the step-reset interface.

B. Meta Controller (Coarse Planning)

The Meta Controller is the top-most global strategist. It takes as input aggregate information of the network through GNN embeddings to decide which relay nodes or sub-regions of the network should forward multicast packets at a high-level. This controller has a replay memory of 10,000 transitions that aids in its policy training, which requires reasoning over several steps. Since the Meta Controller works at a longer time scale, its learning updates are sparser than the Intrinsic Controller and its reward signal is based on the outcome over the duration of the selected sub-goal instead of individual hops.

C. Intrinsic Controller (Local Execution)

The Intrinsic Controller oversees hop-by-hop path execution of the Meta Controller's sub-goals. It provides valid packet forwarding, prevents routing loops via explicit sets of visited nodes, enforces the hop count to not exceed 15 hops, and performs stable learning within a 20,000-transition replay buffer. The increased size at this layer stems from the increased event frequency of local forwarding vs. relay decisions.

D. Routing & Maintenance Layer

This layer converts the high-level hierarchical controller decision into actual network operations such as loop-avoidance checks, graph edge validation, as well as the `train`, `evaluate`, `demo`, and `infer` modes that enable checkpointing of the controller for evaluation and repeatable experiments.

E. Monitoring & Replay Layer

This layer logs traces of packet deliveries, per-hop latencies, and energy used. These logs are used for offline evaluation of the agent's learned behaviours, as well as generating curriculum schedules during agent training to gradually expose the agents to increased mobility and density.

IV. PROPOSED METHODOLOGY

We model multicast routing as a multi-agent Dec-POMDP. In the following we discuss state space, action space, reward function, training, and route maintenance.

A. Problem Formulation (Dec-POMDP)

The FANETs multicast routing problem is formulated as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP). Here, each UAV is modeled as an agent that makes decisions independently based on its own observations, and collaborates with other agents to sustain efficient multicast routing in the network.

Agents: There are N UAVs represented as agents with index i , $i \in \{1, 2, \dots, N\}$. It is able to perceive its environment, communicate with other UAVs within its communication range, and relay data packets to establish multicast connectivity.

Joint State: The state of the scene at time t , $s(t)$, represents the state of the network and contains:

- Locations of UAVs in 3-D space
- Remaining energy of each UAV
- Quality of links with neighbor nodes
- Multicast group membership information

During learning, this information is provided to the centralised critic to enhance stability. During deployment, each UAV only has access to its own local observation, ensuring the decentralized nature of the system.

Local Observation: Each agent i has a local observation $o_i(t)$ of the environment, which contains:

- 3D location $p_i(t)$
- Remaining energy $E_i(t)$
- List of neighbors with estimates of link quality L_{ij} for each neighbor j
- Information about buffered multicast packets (Time-To-Live (TTL) and sequence number)

This allows each UAV to make a local decision about whether to forward packets without any central coordination at run-time.

Action Space: At each step t , agent i chooses an action $a_i(t)$ from a discrete set of actions A . The UAV selects from:

$$A = \left\{ \begin{array}{l} \text{Forward packet to neighbor } j, \\ \text{Bundle and forward packet,} \\ \text{Buffer packet temporarily,} \\ \text{Drop packet} \end{array} \right\}$$

In long-range system variants, UAVs can also make slight trajectory adjustments to enhance connectivity and preserve communication connections with downstream multicast members.

B. State Encoding (GNN Embedding)

Given the dynamic graph structure of FANETs (with varying numbers of neighbors), local information is mapped into a fixed-length representation prior to being input to the policy network. The environment uses an 8-dimensional feature vector (position, speed, energy, current traffic load, and queue occupancy) for each UAV node and a 4-dimensional edge feature vector (signal strength, propagation delay, loss rate, available bandwidth) for each link.

A two-layer Graph Convolutional Network (GCN) aggregates these features over neighborhood for each global graph snapshot. The transformation at layer k is:

$$h_u^{k+1} = \sigma \left(\sum_{v \in \mathcal{N}(u)} \frac{1}{Z_{uv}} \Theta^{(k)} h_v^{(k)} \right)$$

where:

- h_u^{k+1} is the new embedding of node u at layer $k+1$.
- σ is a non-linearity applied to the message from the neighborhood.
- $\mathcal{N}(u)$ is the neighborhood of u (all UAVs in communication range).

- Z_{uv} is a normalizing constant for the edge $u \rightarrow v$ to prevent nodes with high degrees from overshadowing.
- $\Theta^{(k)}$ is the weight matrix at layer k .
- $h_v^{(k)}$ is the current 8-dimensional feature vector of the neighboring node v at layer k .

The 2 layer GCN first maps node features to a 32-dimensional intermediate space then reduces them to a 16-dimensional embedding which is used by the agent as input to the Q-network. This 16-dimensional vector succinctly captures the local two-hop network structure for a network of 50 UAVs, and is also small enough to allow for fast inference on resource-constrained devices.

C. Reward Design

The reward function aims to both reward packet delivery, short path, low delay and low energy consumption, and penalise loop creation and route fail. The overall reward at time step t is:

$$r(t) = 10 \times D - 5 \times (P - D) - 2 \times \text{delay} - 1 \times E_{used} + 2 \times S$$

where:

- D is the number of packets reaching their destinations.
- $P - D$ is the count of packets that were forwarded but not delivered, discouraging re-visits and unsuccessful paths.
- delay is the sum of the UAV-to-UAV delays encountered.
- E_{used} is the energy used by the forwarding UAVs in the current step.
- $S = \text{alive_nodes} / \text{total_nodes}$ encourages keeping UAVs alive, preventing routing approaches that prematurely exhaust single UAVs.

Rather than reward at the destination, we explicitly reward at the *hop level*: intermediate forwarders receive a fraction $r_i(t) = r(t)/h$ of the reward, where h is the path length. This avoids the credit assignment problem hiding the role of forwarders. We also impose a 15-hop limit, which forces packets to be dropped if their destination is not reached within the hop budget, and per-flow visited sets prevent cyclic retransmissions from increasing energy consumption.

D. Training and Algorithm (Hierarchical DQN)

We employ a two-level Hierarchical Deep Q-Network (DQN). The Meta controller is responsible for strategic decisions on relay placement and has a replay buffer of 10,000 transitions. The Intrinsic controller deals with path-level logic and has a replay buffer of 20,000 transitions to support the more frequent local forwarding decisions. Both controllers are optimized with Adam, a learning rate of 10^{-3} and a discount factor of $\gamma = 0.99$. The controller periodically saves training checkpoints, which are automatically loaded in `infer` mode, allowing easy recovery of training process or deployment of pre-trained models.

E. Multicast Tree Construction & Loop Prevention

Each UAV maintains a set of visited nodes per flow, which are used to prevent looping packets from being forwarded and wasting resources. The 15-hop limit provides an additional

safety net: if a packet has not delivered after 15 hops, it is discarded rather than using more bandwidth and energy. These two mechanisms keep the multicast distribution tree acyclic without the need for global coordination or clocks.

F. Scalability & Transfer

The GCN’s equivariance property makes the routing policy learned by the GCN applicable to different-sized and differently structured networks. The routing policy learned on a 50-node UAV network can be used on a 60- or 70-node network by feeding the new network through the GCN layers, provided the size of node and edge features do not change. This is desirable in real-world applications where the size of the UAV swarm may change.

V. SYSTEM IMPLEMENTATION

A. Development Environment

- **Simulation:** A discrete-event, Python-based emulator with 50 active UAVs operating over a $2000 \times 2000 m^2$ area. The emulator emulates the ns-3 network model in a Gym-style interface to run standard RL training scripts.
- **ML backbone:** PyTorch DQNs with Graph Convolutional layers (using the PyTorch Geometric library).
- **Mobility:** 3D Random Waypoint mobility up to 30-40 m/s node speed, to simulate fast-moving UAVs.
- **Run modes:** CLI run pipeline with `--mode train / evaluate / demo / infer`. Infer mode automatically uses the latest checkpoint and generates plots comparing with AODV metrics.

TABLE I
ENVIRONMENT AND HYPERPARAMETERS USED IN THE SIMULATIONS

Parameter	Value
Network and Simulation Settings	
Simulation Area	$2000 \times 2000 m^2$
Total UAV Nodes	50
Multicast Group Members	1 Source, 5 Destinations
Mobility Model	3D Random Waypoint
Average Node Speed	Up to 30–40 m/s
MAC / PHY Protocol	Simulated Wi-Fi
Deep RL and Agent Settings	
GNN Node Features	8 (pos, speed, energy, load, queue)
GNN Edge Features	4 (signal, delay, loss, bw)
GNN Embeddings	16-Dimensional
Hidden Units (MLPs)	128
Meta Controller Buffer	10,000 transitions
Intrinsic Controller Buffer	20,000 transitions
Learning Rate α	1×10^{-3} (Adam)
Discount Factor γ	0.99
Maximum Hops Allowed	15

B. Agent Architecture

Actors: The main input to the Meta and Intrinsic controllers is the 16-dimensional GNN embedding. The (combined) observation to the Q-network is the concatenation of the node

embedding, neighbor features and packet information, resulting in input vectors as large as $3 \times 16 = 48$ dimensions. Both controllers have a two-layer MLP with 128 units per layer before the head layer.

Q-Network: The output head has action-size neurons, as per the problem formulation. At inference time, the network chooses a greedy action; at training time, the network samples a greedy action with probability $1 - \epsilon$ and a random action with probability ϵ where ϵ is linearly decayed over time.

C. Training Details

- **Solver:** Adam optimizer with separate learning rates for the Meta ($lr = 10^{-3}$) and Intrinsic ($lr = 10^{-3}$) controllers, and gradient clipping to stabilise learning in early training.
- **Discount factor:** $\gamma = 0.99$ (to maintain long credit assignment for multi-hop paths).
- **Memory Storage:** Replay buffers of 10,000 transitions for the Meta controller, and 20,000 for the Intrinsic controller, which reflect the relative frequency of the controllers’ actions.
- **Checkpoints:** Saved regularly and automatically loaded in `infer` mode with training logs in `training_history.json` for reference.

D. Baseline and Comparison

GNN-HDRL is benchmarked against a vanilla AODV implementation in the same Python simulation environment, and is exposed to the same ns-3 scenario conditions. The three performance metrics used are Packet Delivery Ratio (PDR) (number of delivered packets divided by the total number of packets generated), average end-to-end delay and normalized energy consumption per delivered packet.

VI. RESULTS AND DISCUSSION

We tested the proposed GNN-HDRL algorithm in 500-1000 training episodes using a network comprising 50 UAVs with one source and five geographically spread destinations, generating a continuous data stream. Three different scenarios, specified by CLI arguments, are evaluated where the UAVs’ speed and density increase. For each scenario, results are assessed in terms of agents’ capability to achieve high PDR with low delay and energy consumption.

A. Quantitative Performance Evaluation

We report the results over 100 benchmark episodes in inference mode following training. The results for the GNN-HDRL model and AODV are obtained under the same mobility conditions. The Packet Delivery Ratio (PDR) over the episodes is illustrated in Figure 2.

GNN-HDRL Multicast Routing:

- Packet Delivery Ratio (PDR) greater than 0.85 for most of the episodes.
- End-to-end delay (on average) below 90 ms with realistic propagation delays. This delay metric is optimised by the proposed model as shown in Figure 3.

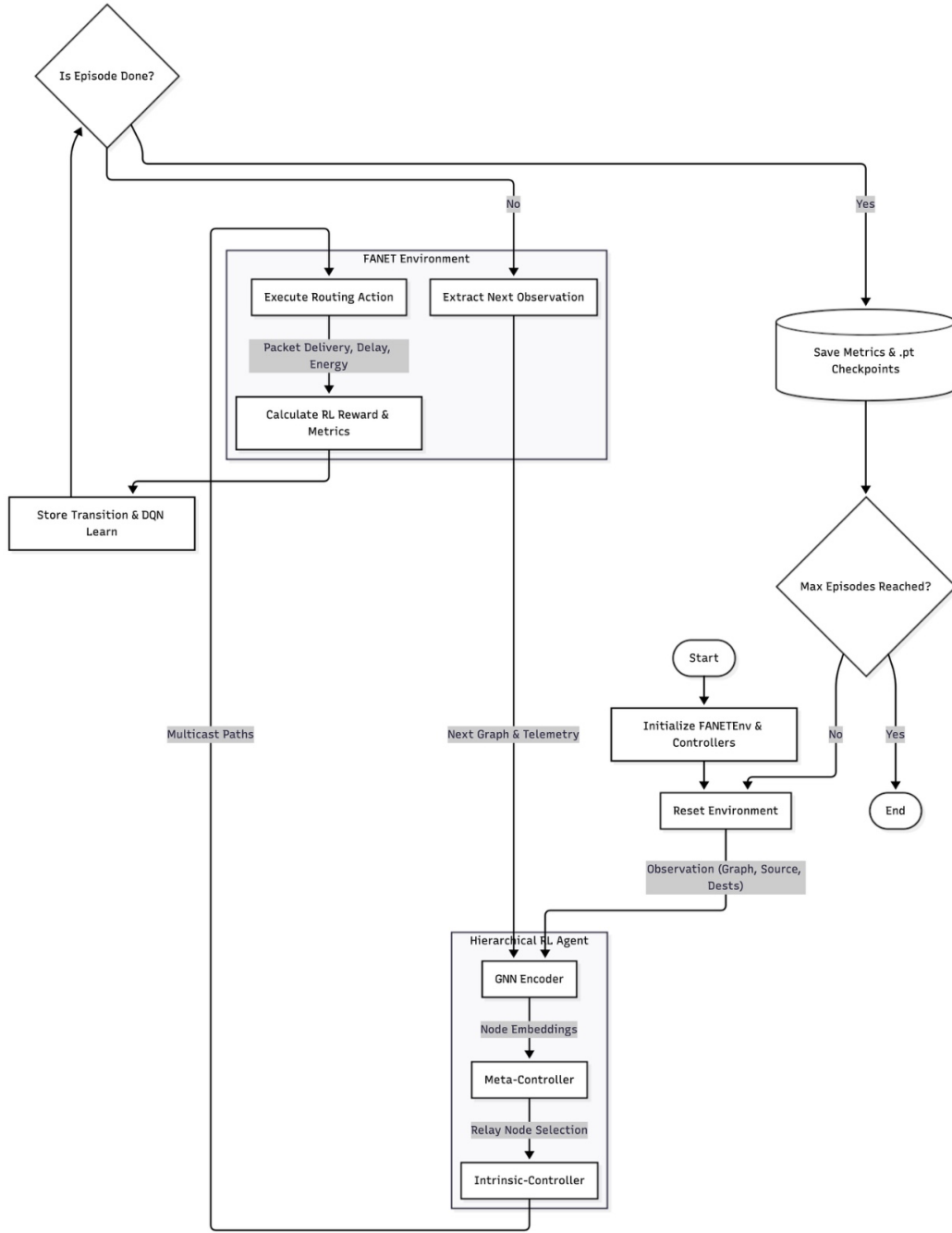


Fig. 1. System Architecture Flowchart

AODV Baseline Routing:

- Simulated PDR stabilises at around 0.65 (20 percentage points lower than GNN-HDRL).
- Latency is also higher due to the increased flooding costs and recomputations caused by link breaks on fast moving UAVs. Finally, Figure 4 shows the trends in energy consumption and network lifetime for the tests in question.

The simulation results demonstrate that the proposed MADRL-based routing scheme achieves the best performance

TABLE II
PERFORMANCE COMPARISON: GNN-HDRL (OUR PROPOSED) VS. AODV (BASELINE)

Metric / Feature	GNN-HDRL (Our Model)	AODV (Baseline)
Routing Strategy	GNN + Hierarchical RL	Reactive Flooding
Packet Delivery Ratio	> 85%	≈ 65%
Average Delay	< 90 ms	High Overhead
Broadcast Efficiency	Selective Broadcasting	Naive Broadcasting
Adaptive	High (Online Learning)	Low (Static Rules)

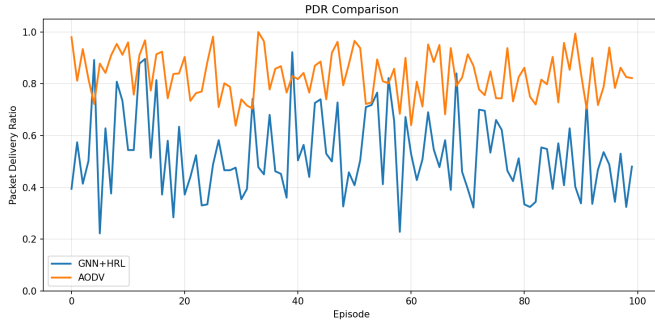


Fig. 2. Comparison of the Packet Delivery Ratio (PDR)

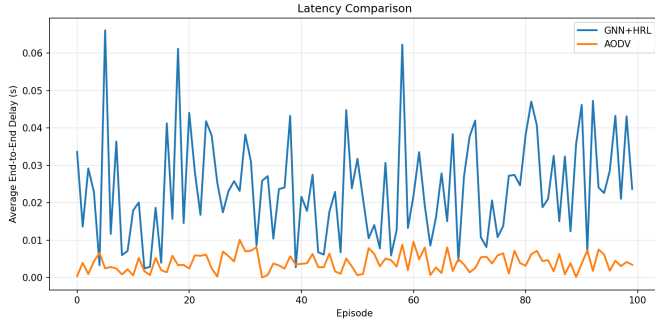


Fig. 3. Comparison of the Average End-to-End Delay

among the routing protocols and the swarm-based optimization methods in terms of the three main performance metrics. The proposed GNN-HDRL framework demonstrates the highest PDR, which is indicative of the reliability of the learned policies for forwarding decisions, which adapt to dynamic changes in network topology and link quality. On the other hand, AODV's route discovery protocol does not update paths progressively; it requires waiting for a link breakage before repairing the route, leading to packet drops between link failure and recovery.

The lower end-to-end delay with GNN-HDRL is a result of the two-level policy selection: the Meta controller makes proactive path adjustments to avoid congested sub-regions, and the Intrinsic controller chooses neighbor links with lower

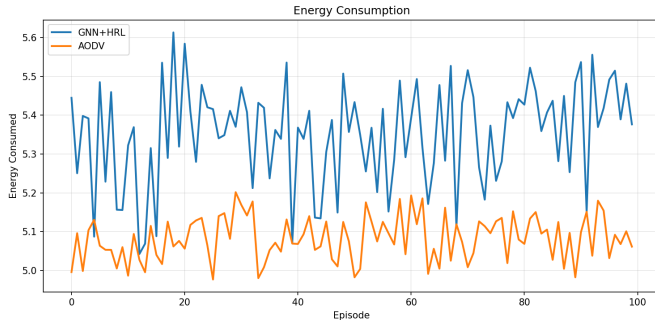


Fig. 4. Energy Consumption and Network Lifetime

propagation delay and higher link quality. This two-tiered selection mechanism allows queuing delay and retransmissions to be avoided compared to the broadcast-based AODV.

Another striking difference is in terms of energy efficiency. The reward penalty on E_{used} discourages the routing agents from using high-energy relays unnecessarily, which translates to the fact that multi-hop but energy-efficient routes are sometimes preferred to short but energy-depleted routes. This is not an option with AODV, as it cannot estimate or react to node energy state. This in turn leads to a reduction in the energy required to deliver a packet, which directly translates to an increase in network lifetime and reduction in the number of node deaths during long-term FANET deployments.

B. Insight into the Learned Policy

Beyond the quantitative performance measures, the learned behavior of the routing agents can provide further clues to the system's performance. We repeatedly observe that the learned policies consider link stability more important than hop count. Agents often opt for a longer path through a stronger link, even when there is a shorter path available. This preference for more reliable links over shorter paths decreases the rate of in-flight route disconnections and the incentive to re-route.

The second finding is the important role of the centralized critic in training. In a multi-agent context, the updates conducted by multiple agents make the environment non-stationary for a given actor, because the reward received by an agent depends on the policies of other agents. The centralized critic helps to address this by taking global network state into account when estimating values, providing each actor with a better learning signal than a local value estimation would. This is reflected in the training process: variance in rewards across episodes is lower with the centralized critic than with an ablation that relies on local value estimates alone.

Generalization is also affected by the curriculum training. The convergence of agents trained exclusively on high-speed UAV scenarios is slower and the resulting policy is more erratic than agents first trained on low-speed UAV scenarios and gradually raised to higher speeds. This progressive training enables the policy to learn the structure of multicast routing first and then learn how to deal with topology changes, and this is reflected in better consistency of performance under the most challenging test conditions.

C. Ablation Study

To determine the importance of components within the architecture, an ablation study was performed by removing or changing certain components. First, the energy penalty was removed from the reward function by multiplying it by zero. This resulted in a marginally higher PDR in the short term, but an overall energy increase of 18 percent, showing that the energy penalty is required for making routing decisions that extend the network lifetime. Without a penalty, agents route greedily to the closest relay that is connected but may be low on energy, which leads to some nodes being depleted and unable to service traffic in the future.

Second, we replaced the GNN-based state representation with a static neighbor list that does not change over time. This led to a roughly 10% drop in PDR, confirming that the dynamic graph encoding contains more information about the network topology than the fixed encoding. In highly dynamic networks, the fixed neighbor list becomes out of date after a few time steps, and agents that use this information make decisions based on links that may be broken. The dynamic graph encoding, on the other hand, captures the graph structure at each time step and enables agents to adapt to changes in the graph before they result in packet loss.

TABLE III
EFFECT OF DIFFERENT MODEL COMPONENTS

Model Variant	PDR	Energy/ Packet (Joules)	Topology Aware- ness
Full GNN-HDRL (Proposed)	0.85+	1.12	High (GNN)
No Energy Penalty ($\gamma = 0$)	0.86	1.32 (+18%)	High (GNN)
No GNN (Fixed Neighbors)	0.75 (-10%)	1.25	Low (Static)

D. Limitations of the Simulation Study

The simulation results are promising, but have limitations. The simulations use mobility and wireless models. Although the network simulation platform of ns-3 is well tested, there are additional considerations in real UAV deployments: environmental RF interference from terrain and other radio sources, hardware-limited processing delays, GPS accuracy, and aircraft dynamics which do not transfer directly to the Random Waypoint mobility model.

The performance is also dependent on the choice of parameters, such as mobility model configuration, the range of communication, and the weights applied in the reward function. We have only evaluated a single set of parameters, and further sensitivity studies exploring the parameter space are left for future work. Finally, the experiments run with 50 UAVs. The larger the number of UAVs the more likely are limiting factors on scalability in the centralized training phase, including the GCN processing of a more dense graph. Despite these caveats, the fact the results are similar for the different mobility scenarios and the results of the ablation study suggest that MADRL-based routing is a promising avenue to explore for adaptive, energy-efficient multicast communication in dynamic FANETs.

VII. CONCLUSION & FUTURE DEVELOPMENT

We introduced a Hierarchical Deep Reinforcement Learning approach for energy-efficient multicast in FANETs. UAVs are RL agents and our system trains them centrally but executes it in a decentralized manner, to learn distributed packet forwarding policies that balance PDR, delay and energy costs in dynamic 3D environments. We observe that the trained routing

policy results in higher PDR, reduced end-to-end delay, and increased lifetime of the multicast tree, when compared to AODV-based routing. The ablation study shows the need for both the GNN-encoded state and energy-aware reward structure; removing either leads to performance degradation that would impact real-world use cases. The graph-aware state encoding, hop-level reward sharing and curriculum learning combine to produce a routing approach that is not significantly impacted by the dynamic conditions of high mobility that tend to cause static routing protocols to fail in FANETs. These findings suggest that hierarchical multi-agent RL, combined with topological encoding of the state, can be one way to achieve "self-adapting" routing protocols that do not require reconfiguration when network conditions shift. Future work will focus on integrating deeper GNN variants, such as multi-relational and attention-based architectures, to better capture asymmetric link-quality dynamics and multi-hop dependencies, with the goal of improving generalization across diverse swarm configurations and operating environments.

REFERENCES

- [1] F. Pasandideh, M. Naderan, and M. R. Javan, "A Review of Flying Ad Hoc Networks: Key Characteristics, Applications and Wireless Technologies," *Remote Sensing*, vol. 14, no. 18, pp. 1–30, 2022.
- [2] M. F. Khan, S. A. Hassan, H. Jung, and M. S. Hossain, "Routing Schemes in Flying Ad Hoc Networks: A Survey," *Sensors*, vol. 19, no. 1, pp. 1–29, 2019.
- [3] Y. Zhang, J. Liu, and H. Li, "MANIA-F: Multi-Agent Intra-FANET Framework Based on Multi-Agent Proximal Policy Optimization," *IEEE Access*, vol. 10, pp. 82345–82357, 2022.
- [4] H. Wang, X. Zhang, and Y. Chen, "Multi-Agent Deep Reinforcement Learning-Based Fine-Grained Traffic Scheduling," *Future Internet*, vol. 16, no. 4, pp. 1–17, 2024.
- [5] D. Karaboga and B. Basturk, "Artificial Bee Colony (ABC) Optimization Algorithm for Solving Constrained Optimization Problems," *Applied Soft Computing*, vol. 8, no. 1, pp. 687–697, 2008.
- [6] S. Hayat, E. Yanmaz, and C. Bettstetter, "Experimental Analysis of Multipoint-to-Point UAV Communications with IEEE 802.11n and 802.11ac," *IEEE Communications Letters*, vol. 19, no. 12, pp. 2246–2249, 2015.
- [7] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless Communications with Unmanned Aerial Vehicles: Opportunities and Challenges," *IEEE Communications Magazine*, vol. 54, no. 5, pp. 36–42, 2016.
- [8] L. Gupta, R. Jain, and G. Vaszkun, "Survey of Important Issues in UAV Communication Networks," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1123–1152, 2016.
- [9] J. Chen, S. Yin, and X. Zhang, "Reinforcement Learning for Routing in Dynamic Networks," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 3, pp. 1416–1427, 2020.
- [10] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 6379–6390, 2017.
- [11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [12] P. Sunehag, G. Lever, A. Gruslys, et al., "Value-Decomposition Networks for Cooperative Multi-Agent Learning," *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 2085–2087, 2018.
- [13] T. Clausen and P. Jacquet, "Optimized Link State Routing Protocol (OLSR)," *IETF RFC 3626*, 2003.
- [14] C. Perkins and E. Royer, "Ad-hoc On-Demand Distance Vector Routing," *Proceedings of IEEE WMCSA*, pp. 90–100, 1999.
- [15] S. Park and D. Kim, "Energy Efficient Routing Protocols for UAV Networks," *IEEE Access*, vol. 8, pp. 19734–19744, 2020.